

LATIHAN SOAL

Pohon Biner

KAMUS UMUM

{ deklarasi dan inisiasi konstanta global }

constant NIL = NULL

{ deklarasi Abstract Data Type }

type boolean: (true, false)

type ElType: integer

type BinTree : pointer to TreeNode

type TreeNode : < info : ElType,
 left : BinTree,
 right : BinTree >

IMPLEMENTASI FUNGSI

function search (P: BinTree, X: ElType) → boolean

{ Mengirimkan true jika ada node dari P yang bernilai X }

KAMUS LOKAL

{ Tidak ada kamus lokal }

ALGORITMA

depend on (P):

(P = NIL):

→ false

(P↑.info = x):

→ true

(P↑.info ≠ x):

→ search(P↑.left, x) or search(P↑.right, x)

function isSkewLeft (P: BinTree) → boolean

{ Mengirimkan true jika P adalah pohon condong kiri }

{ Pohon condong kiri: setiap node internal hanya memiliki anak kiri }

KAMUS LOKAL

{ Tidak ada kamus lokal }

ALGORITMA

```
depend on (P):  
  (P = NIL):  
    → true  
  (P↑.right ≠ NIL):  
    → false  
  (P↑.right = NIL):  
    → isSkewLeft(P↑.left)
```

function isSkewRight (P: BinTree) → boolean

{ Mengirimkan true jika P adalah pohon condong kanan }
{ Pohon condong kanan: setiap node internal hanya memiliki anak kanan }

KAMUS LOKAL

{ Tidak ada kamus lokal }

ALGORITMA

```
depend on (P):  
  (P = NIL):  
    → true  
  (P↑.left ≠ NIL):  
    → false  
  (P↑.left = NIL):  
    → isSkewRight(P↑.right)
```

function level (P: BinTree, X: ElType) → integer

{ Mengirimkan level dari node X yang merupakan salah satu daun dari pohon biner P. Akar(P) level-nya adalah 1. Pohon P tidak kosong dan elemen-elemennya unik }

KAMUS LOKAL

{ Tidak ada kamus lokal }

ALGORITMA

```
depend on (P):  
  (P↑.info = x):  
    → 1  
  (search(P↑.left, x)):  
    → 1 + search(P↑.left, x)  
  (search(P↑.right, x)):  
    → 1 + search(P↑.right, x)
```

```

procedure addDaun (input/output P: BinTree, input x, y: ElType,
                    input Kiri: boolean)
{ I.S. P tidak kosong, X adalah daun Pohon Biner P
  F.S. P bertambah simpulnya, dengan Y sebagai anak kiri X (jika
    Kiri), atau sebagai anak kanan X (jika not Kiri). Jika ada
    lebih dari satu daun bernilai X, Y ditambahkan pada daun
    paling kiri }

```

KAMUS LOKAL

```

{ deklarasi prosedur helper }
procedure addDaunHelper(input/output P: BinTree,
                        input x, y: ElType,
                        Kiri: boolean,
                        input/output found: boolean)

found: boolean

```

ALGORITMA

```

found ← false
addDaunHelper(P, x, y, Kiri, found)

```

[⚠] Implementasi Fungsi addDaunHelper untuk mengatasi permasalahan variabel found yang "hidup" independen di tiap rekursif

```

procedure addDaunHelper(input/output P: BinTree, input x, y: ElType,
                        Kiri: boolean, input/output found: boolean)
{ Fungsi helper untuk mengatasi permasalahan variabel found
  yang "hidup" independen di tiap rekursif }

```

KAMUS LOKAL

```

{ Tidak ada kamus lokal }

```

ALGORITMA

```

if (P ≠ NIL and not found) then
  if (P↑.info = x and P↑.left = NIL and P↑.right = NIL) then
    if (Kiri) then
      P↑.left ← newTreeNode(y)
    else
      P↑.right ← newTreeNode(y)
    found ← true
  else
    addDaunHelper(P↑.left, x, y, Kiri, found)
    addDaunHelper(P↑.right, x, y, Kiri, found)

```

procedure delDaun(**input/output** P : BinTree, **input** x : ElType)

{ I.S. P tidak kosong, minimum 1 daun bernilai X
F.S. Semua daun yang bernilai X dihapus dari P }

KAMUS LOKAL

{ Tidak ada kamus lokal }

ALGORITMA

if (P↑.info = x **and** P↑.left = NIL **and** P↑.right = NIL) **then**

 deallocTreeNode(p)

else

if (P↑.left ≠ NIL) **then**

 delDaun(P↑.left, x)

if (P↑.right ≠ NIL) **then**

 delDaun(P↑.right, x)